

Sign up

Sign in

Medium



Image Enhancement and OCR: A Practical Guide to Improve Text Extraction

11 min read · Oct 9, 2025



Shourya

Follow



Listen



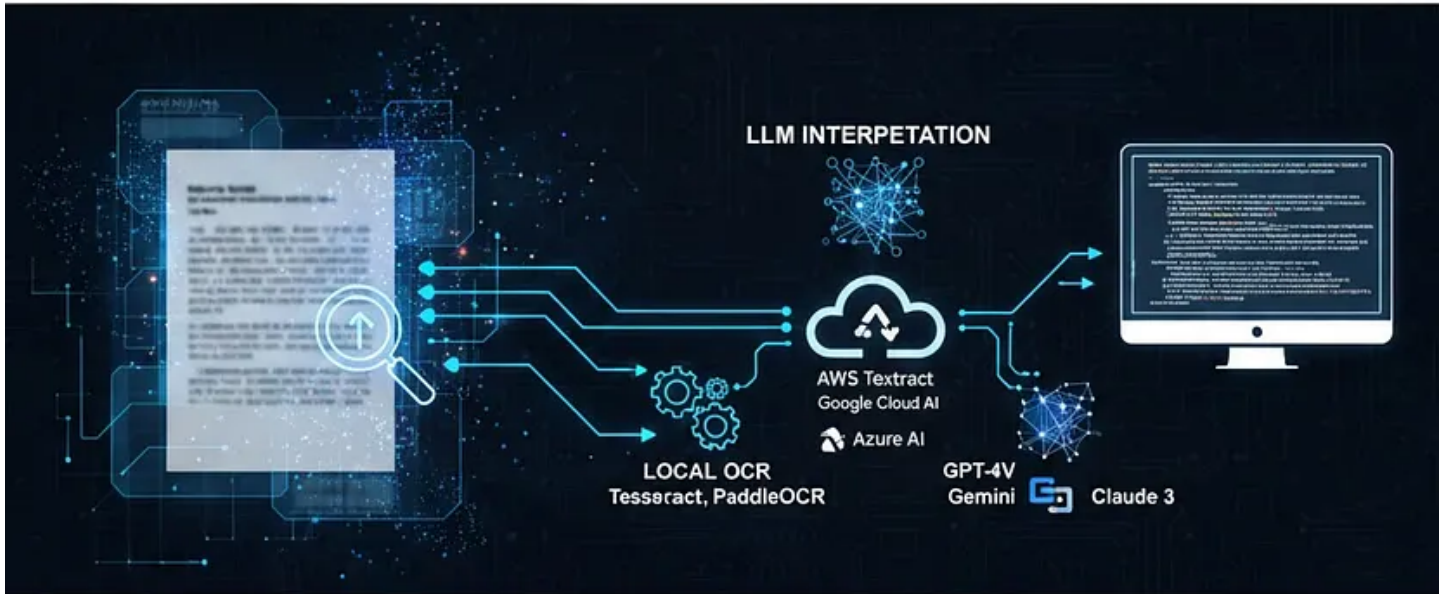
Share



14



IMAGE ENHANCEMENT AND OCR: A Practical Guide to Improve Text Extraction



Introduction

This guide shares the practical techniques I have used to improve text extraction from images. It covers a straightforward process: first, correcting the image to make it proper for OCR to read the text, and second, selecting the right OCR tool for the job.

I will walk through the specific steps that have consistently delivered better results

for me, such as correcting image orientation, adjusting resolution, and cropping to focus on important areas. I will also compare different OCR models — from open-source Python libraries to commercial services to LLMs— and explain when to use each one.

Before feeding an image to an OCR model, cleaning it up is the single most effective way to improve accuracy. While many pixel-level adjustments like brightness and contrast exist, I've found their impact can be limited. Instead, this section focuses on three techniques that have consistently made a significant difference in my projects: resolution changes, orientation correction, and strategic cropping.

1.1 Resolution Change

Purpose: To ensure text is large and clear enough for the OCR model to recognize individual characters. Low-resolution images often cause characters to appear blurry or merged, leading to high error rates.

Recommended ranges: Aim for a resolution where the smallest characters are at least 15–20 pixels tall. For scanned documents, a target of 300 DPI (dots per inch) is a common best practice. For an image, resolution between 3000–5000 is good to have.

When to upscale vs avoid: Upscale when the original image has very small or pixelated text. Avoid blindly upscaling already clear, high-resolution images, as it adds processing time without benefit and can sometimes amplify noise.

Example workflow: Here is a simple example of using OpenCV to increase image resolution. In one of my tests, this change increased the amount of extracted text from 433 characters to 1858 characters, showing a dramatic improvement in detection.

```
import cv2
import numpy as np

def change_image_resolution_cv2(input_path: str) -> np.ndarray:
```

```
"""
Change the resolution of an image using OpenCV.
"""
img = cv2.imread(input_path)
resized_img = cv2.resize(img, (5000, 5000), interpolation=cv2.INTER_AREA)
return resized_img

# Example Result:
# Original Image Text Extracted Length: 433
# Resized Image Text Length: 1858
```

Original Image vs Resized Image

Challenges / trade-offs: Aggressive resizing can distort the image's aspect ratio. It's often better to scale by a factor (e.g., 2x) rather than forcing a fixed dimension.

1.2 Orientation Correction

Common issues (rotated scans, mobile capture): OCR models expect text to be horizontally aligned (either 0 or 180 deg). Images that are skewed or rotated will almost always fail OCR.

Detection methods (heuristics vs model-based): You can use libraries like *deskew* to programmatically determine the skew angle of the text.

Correction steps: Once the angle is determined, use a library like OpenCV to rotate the image back to a horizontal alignment.

Validation approach: The corrected image should have text lines that are perfectly horizontal. A simple visual check or re-running OCR will confirm the fix.

Example workflow:

```
import math
from typing import Tuple, Union

import cv2
import numpy as np

from deskew import determine_skew
# Function to calculate orientation of image
def calculate_orientation(image: np.ndarray) -> float:
    """Calculate the orientation of text in the image."""
    try:
        if len(image.shape) == 3:
            gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        else:
            gray_image = image
        angle = determine_skew(gray_image, angle_pm_90=True)
        return angle
    except Exception as e:
        return None

def rotate(
    image: np.ndarray, angle: float, background: Union[int, Tuple[int, int, int]]
) -> np.ndarray:
    old_width, old_height = image.shape[:2]
    angle_radian = math.radians(angle)
    width = abs(np.sin(angle_radian) * old_height) + abs(np.cos(angle_radian) *
    height = abs(np.sin(angle_radian) * old_width) + abs(np.cos(angle_radian) *

    image_center = tuple(np.array(image.shape[1::-1]) / 2)
    rot_mat = cv2.getRotationMatrix2D(image_center, angle, 1.0)
```

```
rot_mat[1, 2] += (width - old_width) / 2
rot_mat[0, 2] += (height - old_height) / 2
return cv2.warpAffine(image, rot_mat, (int(round(height)), int(round(width))

# image = cv2.imread('image.png')
# grayscale = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# angle = calculate_orientation(grayscale)
# rotated = rotate(image, angle, (0, 0, 0))
# cv2.imwrite('output.png', rotated)
```

orientation correction before vs after

1.3 Cropping (Segmentation / Tiling)

Why segment large or complex images: For very large images or documents with dense, multi-column layouts, processing the entire image at once can be inefficient. Tiling breaks the image into smaller, manageable chunks.

Approaches (manual ROI, automatic detection, tiling): Some OCR engines, like PaddleOCR, have built-in support for automatic tiling (slicing). You can define a stride and the model will process overlapping parts of the image.

Impact on latency and accuracy: This technique can significantly improve accuracy on dense documents. In one test, using PaddleOCR's slicing feature doubled the extracted text length of characters.

Reassembly considerations: When using a built-in feature like PaddleOCR's, the engine handles merging the results from the different tiles back into a coherent text block.

Example Workflow:

```
# paddleocr==2.10.0
# paddlepaddle==3.1.0
from paddleocr import PaddleOCR
import cv2

# Initialize PaddleOCR model
model = PaddleOCR(use_angle_cls=True, lang='en') # Initialize PaddleOCR model

image = cv2.imread('image.png')
# Without Cropping
result = model.ocr(image, cls=True) # Perform OCR
result = result[0] if result[0] else []
text = "\n".join([line[1][0] for line in result])

print("Extracted Text Before Cropping:\n", text)
print("length of text before cropping:", len(text))

# With Cropping
height, width = image.shape[:2]
result = model.ocr(image, cls=True, slice={'horizontal_stride': width // 3, 'v
result = result[0] if result[0] else []
text = "\n".join([line[1][0] for line in result]) # Extract text from results

# length of text before cropping: 3957
# length of text after cropping: 4217
```

2. Best Python-OCR Models

Introduction: You might have experimented with OCR libraries like Tesseract,

EasyOCR, or Keras-OCR. While these have their uses, Rapid OCR and PaddleOCR currently stand out for their capabilities and active development.

2.1 Rapid OCR

Strengths: Lightweight, fast, and optimized for edge devices. Good for quick text extraction tasks where speed is critical.

Limitations: May not be as accurate as more complex models on challenging images (e.g., low resolution, complex layouts).

Ideal use cases: Real-time OCR on mobile devices, extracting text from video frames, or batch processing large numbers of simple documents.

```
# pip install rapidocr-onnxruntime
from rapidocr_onnxruntime import RapidOCR

rapid_ocr = RapidOCR()
result, _ = rapid_ocr("your_image.jpg")
print(result)
# Official Documentation: https://rapidai.github.io/RapidOCRDocs/main/
```

2.2 Paddle OCR

Strengths: Highly accurate, supports multiple languages, and offers a flexible pipeline with detection, classification, and recognition components.

Limitations: Larger and more resource-intensive than Rapid OCR.

Model components (detection, classification, recognition): PaddleOCR uses a DBNet for text detection, a CRNN or Transformer-based model for text recognition, and a text classifier to determine text orientation.

Get Shourya's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

When to select over alternatives: Choose PaddleOCR when accuracy is paramount, you need to support multiple languages, or you are working with complex document layouts.

```
# pip install paddleocr
from paddleocr import PaddleOCR

ocr = PaddleOCR(use_angle_cls=True, lang='en') # need to run only once to download
img_path = 'your_image.jpg'
result = ocr.ocr(img_path, cls=True)
for line in result:
    print(line)
# Official Documentation: https://www.paddleocr.ai/main/en/index.html
```

3. Commercial Models

Commercial OCR services are designed for organizations that need robust, scalable, and highly accurate text extraction — often beyond what open-source tools can provide. These managed solutions offer advanced features such as handwriting recognition, table and form extraction, and compliance with industry regulations. They are particularly valuable when processing large volumes of documents, integrating OCR into enterprise workflows, or meeting strict security and privacy requirements. In this section, we'll compare the leading commercial OCR offerings

from Microsoft Azure, Amazon Web Services, and Google Cloud, highlighting their strengths, limitations, and ideal use cases.

3.1 GCP: Document AI and Cloud Vision API

GCP offers two main services for OCR, depending on the complexity of the document.

Service Name: Document AI (for structured documents like forms, invoices) and Cloud Vision API (for general image OCR)

Core Features: Document AI offers specialized pre-trained parsers (e.g., Invoice Parser, W-2 Parser), and the Document OCR Processor for raw text extraction. It also provides a Workbench for building custom parsers. The Vision API's Text Detection is better for simple images.

Key Advantage: High-accuracy specialized extraction from complex documents (Document AI) and excellent general-purpose image OCR (Vision API).

Pricing Model (Document OCR): Pay-per-page model. The Document OCR processor is generally priced higher than the Vision API but lower than specialized parsers. Example: First 1 million pages are *\$0.60 to \$0.70 per 1,000 pages* for the Document OCR processor, with the price decreasing at higher volumes.

Pricing Model (Vision API): Pay-per-feature and per-image model. Example: Text Detection is free for the first *1,000 units/month*, then *\$1.50 per 1,000 units* for the next *4,000,000*.

Sample Python Code (Vision API — Basic OCR): See below.

This code detects and extracts text from an image file stored locally.

```
from google.cloud import vision
import io
```

```
def detect_text(path):
    """Detects text in the image file."""
    client = vision.ImageAnnotatorClient()

    with io.open(path, 'rb') as image_file:
        content = image_file.read()

    image = vision.Image(content=content)

    response = client.text_detection(image=image)
    texts = response.text_annotations

    if texts:
        # The first annotation contains the entire document text
        print(f'Full Text:\n{texts[0].description}')
    else:
        print('No text detected.')

    # Example usage:
    # detect_text('path/to/your/image.jpg')
```

3.2 Azure: AI Vision and Document Intelligence

Azure offers both AI Vision and Document Intelligence for OCR, each suited for different tasks.

Service Name: Azure AI Vision and Azure AI Document Intelligence

Core Features:

AI Vision: Offers OCR for general image text extraction.

AI Document Intelligence: Provides advanced document processing, including layout analysis, table extraction, and prebuilt models for invoices, receipts, and IDs.

Pricing Model: Pay-per-page model, with different prices for the basic OCR (Read model) and the advanced analysis models. Example: The Read model (basic OCR) is typically around \$0.50 to \$1.00 per 1,000 pages for lower volumes, with tiered discounts. Prebuilt and Custom models are generally more expensive.

Integration with Azure AI: Azure AI Document Intelligence can be integrated with other Azure AI services to perform further analysis on the extracted text, such as summarization, translation, or entity recognition. This allows for more advanced document processing workflows.

Sample Python Code (Basic OCR): See below.

```
from azure.core.credentials import AzureKeyCredential
from azure.ai.documentintelligence import DocumentIntelligenceClient

# Set your Document Intelligence endpoint and key
endpoint = "YOUR_DOCUMENT_INTELLIGENCE_ENDPOINT"
key = "YOUR_API_KEY"

def analyze_document_read(document_url):
    """Analyzes a document using the prebuilt-read model (basic OCR)."""
    document_intelligence_client = DocumentIntelligenceClient(
        endpoint=endpoint, credential=AzureKeyCredential(key)
    )

    # Use 'prebuilt-read' for general-purpose OCR
    poller = document_intelligence_client.begin_analyze_document_from_url(
        "prebuilt-read", document_url
    )
    result = poller.result()

    print("----Document Text----")
    print(result.content)

    # You can also get line and word bounding box information from result.pages

# Example usage (replace with your document's URL):
# analyze_document_read("https://myaccount.blob.core.windows.net/testdocuments/
```

3.3 AWS: Amazon Textract

AWS's dedicated OCR service is *Amazon Textract*, which is an ML-powered service specifically for document and form analysis.

Service Name: Amazon Textract

Core Features: Detect Document Text (basic OCR), Analyze Document (which includes key-value pairs/forms and table extraction), and specialized APIs like Analyze Expense (for receipts and invoices) and Analyze ID (for ID documents). Supports synchronous and asynchronous processing.

Key Advantage: Deep structural analysis and data extraction are built-in and highly capable, especially for forms and tables. It natively returns structured data (key-value pairs) in its response, which is a significant time saver.

Pricing Model: Pay-per-page model, with different rates for plain text extraction and structured data extraction. Example: Detect Document Text (basic OCR) is typically around *\$1.50 per 1,000 pages* for the first 1 million pages. Analyze Document (forms/tables) is more expensive, often *\$15 to \$20 per 1,000 pages*, with tiered discounts.

Sample Python Code (Basic OCR): See below.

```
import boto3

# Set your AWS region
region_name = 'us-east-1'

def detect_text_from_s3(bucket_name, document_key):
    """
    Detects text in a document stored in S3 using Amazon Textract.
    """
    client = boto3.client('textract', region_name=region_name)

    response = client.detect_document_text(
        Document={
            'S3Object': {
                'Bucket': bucket_name,
                'Name': document_key
            }
        }
    )
```

```
)

# Extract all text lines
full_text = []
for block in response['Blocks']:
    if block['BlockType'] == 'LINE':
        full_text.append(block['Text'])

print("----Document Text----")
print('\n'.join(full_text))

# Example usage (you must have an image/PDF uploaded to S3):
# bucket = 'your-s3-bucket-name'
# key = 'path/to/document.png'
# detect_text_from_s3(bucket, key)
```

Summary Differences

OCR comparison on different cloud services

4. OCR Using LLMs

Role of vision-language models: *Large Language Models (LLMs)* with multi-modal capabilities are increasingly used for OCR tasks. They can not only extract text but also understand the context and relationships between different elements in an image.

Best suited tasks (semantic validation, ambiguous fields, table normalization): *LLMs excel at tasks like semantic validation of extracted text, resolving ambiguities, and normalizing tabular data.* They can also be used to extract specific information from documents based on prompts.

Limitations (cost, determinism, hallucination risk): Using LLMs for OCR can be expensive, especially for large volumes of documents. The results may not always be deterministic, and there is a risk of hallucination (generating incorrect or nonsensical information). Also bounding box and accuracy score is difficult to extract from LLMs.

Integration pattern (base OCR → confidence filtering → LLM enrichment): A common pattern is to use a traditional OCR engine (like PaddleOCR or Tesseract) for initial text extraction, then use an LLM to validate and enrich the results.

Risk mitigation (bounding box grounding, explicit prompts): To mitigate the risks, it's important to provide the LLM with clear and explicit prompts, and to ground the extracted text with bounding box information from the OCR engine.

Example with Google AI Studio and Gemini: Google provides Google AI Studio to test Gemini models. Here's an example of how to use the Gemini model to extract text from an image using prompting:

```
# reference https://ai.google.dev/ to create ur own api key
from google import genai
from google.genai import types

# Replace with your actual image path
```

```

with open('path/to/your/image.png', 'rb') as f:
    image_bytes = f.read()

# Please apply changes as per your need in the prompt
prompt = """
You are a highly skilled OCR refinement specialist.
Your task is to extract and correct text from a given image, ensuring accuracy

1. **Image Context:** [Specify the image content, e.g., a scanned legal document]

2. **Instructions:**
    * Transcribe all discernible text from the image with meticulous attention
    * Rectify any OCR-induced errors, including misspellings or character substitutions
    * Format the extracted text for optimal readability.
    * Identify and extract key information fields where applicable (e.g., dates, names)
    * Provide a confidence score (0-100) indicating the reliability of the extraction

3. **Output Specification:**
    ```json
 {
 "corrected_text": "[The complete, accurately transcribed and corrected text]",
 "extracted_fields": {
 "[Field Name 1]": "[Extracted Value 1]",
 "[Field Name 2]": "[Extracted Value 2]",
 ...
 },
 "confidence_score": [Value between 0-100],
 "justification": "[A concise explanation of the confidence score, highlighting areas of uncertainty]"
 }
    ```

4. **Grounding Data (Example - Adapt to your specific image; provide actual bounding boxes for key terms):**
    * Term: "Account Number", Location: Top-Left Corner, Coordinates: (x1, y1, x2, y2)
    * Term: "Due Date", Location: Middle-Right, Coordinates: (x3, y3, x4, y4)
    * ...

Given the image and the above specifications, perform the OCR task and return the result in the following JSON format:
"""

```

```

# Replace with your Gemini API key
client = genai.Client(api_key="YOUR_API_KEY")
response = client.models.generate_content(
    model='gemini-2.5-flash', #Or any other Gemini Model
    contents=[
        types.Part.from_bytes(
            data=image_bytes,

```



```
        mime_type='image/png',  
    ),  
    prompt  
]  
)  
  
print(response.text)
```

Final Thoughts

Improving OCR outcomes is not about relying on a single tool or technique, but about building a thoughtful, repeatable workflow. By systematically enhancing images, choosing the right OCR engine for your needs, and leveraging post-processing (including LLMs when appropriate), you can dramatically reduce errors and manual correction. The landscape of OCR is evolving rapidly, with new models and services emerging that make high-accuracy extraction more accessible than ever.

Start small: benchmark your current process, introduce one enhancement at a time, and measure the impact. Over time, these incremental improvements will add up to a robust, reliable pipeline that saves time and delivers better results. Stay curious, keep experimenting, and share your findings — the field benefits from practical, real-world insights.



Follow

Written by Shourya

5 followers · 5 following

